

Computing Foundations

ACRONYMS

ADT	Abstract Data Type
AI	Artificial Intelligence
ANSI	American National Standards Institute
AVL Tree	Adelson-Velskii and Landis Tree
BCNF	Boyce-Codd Normal Form
BST	Binary Search Tree
CASE	Common Application Service Element
CDRAM	Cache DRAM
CERT	Computer Engineering Response Team
CISC	Complex Instruction Set Computer
CRUD	Create, Read, Update, Delete
CUDA	Compute Unified Device Architecture
DAG	Directed Acyclic Graph
DAL	Database Access Language
DAS	Direct Access Storage
DBCS	Double Byte Character Set
DCL	Data Control Language
DDL	Data Definition Language
DDR SDRAM	Double Data Rate SDRAM
DKNF	Domain/Key Normal Form
DMA	Direct Memory Access
DML	Data Manipulation Language
EDW	Enterprise Data Warehouse
FCFS	First Come, First Served
FIFO	First In, First Out

FPU	Floating Point Unit
HCI	Human-Computer Interface
HMPP	Hybrid Multicore Parallel Programming
HTTP	Hyper Text Transfer Protocol
IPC	Inter-Process Communication
ISA	Instruction Set Architecture
MIMD	Multiple Instruction, Multiple Data Stream
MISD	Multiple Instruction, Single Data Stream
MISRA	Motor Industry Software Reliability Association
ML	Machine Learning
NAS	Network Access Storage
OSI	Open Systems Interconnection
PDU	Protocol Data Unit
RDBMS	Relational DBMS
RDM	Runtime Database Manager
RDRAM	Rambus DRAM
RISC	Reduced Instruction Set Computer
RTOS	Real Time Operating System
SAN	Storage Area Network
SASE	Specific Application Service Element
SDRAM	Synchronous DRAM
SEI	Software Engineering Institute
SIMD	Single Instruction, Multiple Data Stream
SISD	Single Instruction, Single Data Stream
SQL	Structured Query Language
SRTF	Shortest Remaining Time First

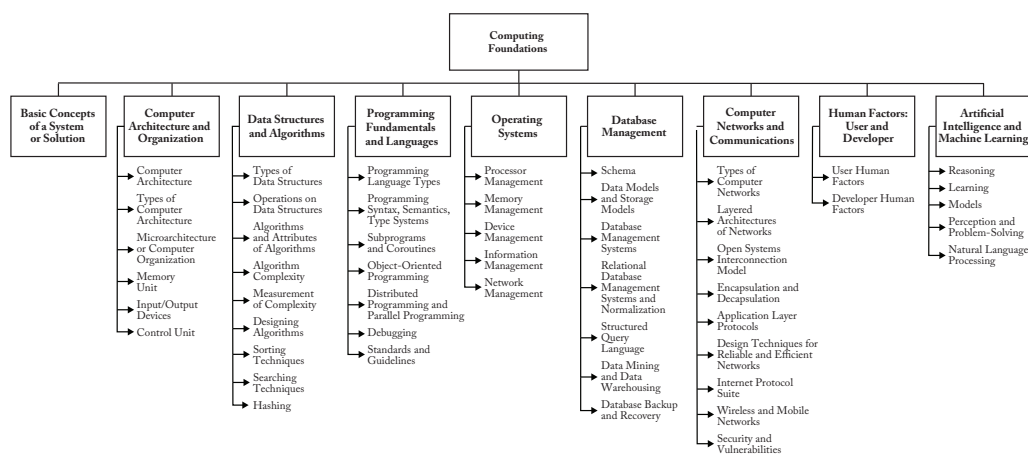


Figure 16.1. Breakdown of Topics for the Computing Foundations KA

INTRODUCTION

Software engineers must internalize the differences between their role and that of a computer programmer. A typical programmer converts a given algorithm into a set of computer instructions, compiles the code, creates links with relevant libraries, binds, loads the program into the desired system, executes the program, and generates output.

On the other hand, a software engineer studies the requirements, architects and designs major system blocks, identifies optimal algorithms, communication mechanisms, performance criteria, test and acceptance plans, maintenance methodologies, engineering processes and methods appropriate to the applications and so on.

The key purpose of the *Software Engineering Body of Knowledge (SWEBOK) Guide* is to identify the areas of knowledge that professional software engineers must know, according to practicing subject matter experts worldwide.

Software engineers are expected to have deep and broad knowledge of various concepts of computer science and be able to apply them. These concepts form the foundations of computing.

BREAKDOWN OF TOPICS FOR COMPUTING FOUNDATIONS

The breakdown of topics for the Computing Foundations knowledge area (KA) is shown in Figure 16.1.

1. Basic Concepts of a System or Solution [6*, C10]

The problem to be solved has to be analyzed in greater detail for functional requirements, user interactions, performance requirements, device interfaces, security, vulnerability, durability and upgradability. A system is an integrated set of subsystems, modules and components that perform specific functions independently. Delineating the problem and solution is critical.

An engineered system ensures the subsystems are designed to be:

- **Modular:** Each subsystem (module) is uniform (similar size).
- **Cohesive:** Each subsystem performs one specific task. Ideally, systems should be highly cohesive.
- **Coupled:** Each subsystem functions independently, as much as possible. Ideally, systems should be loosely coupled.

The subsystems may further be broken down into modules and sub-modules that also exhibit these characteristics.

The system may include both software and hardware subsystems. The hardware must be designed to support the software subsystems and satisfy all user requirements, especially user interfaces (input/output (I/O)) and performance.

This section focuses on designing and building engineered software subsystems.

The applications may require systems that are manual or fully or semiautomated; real-time, online or offline; distributed or single-location, and so on.

The software subsystems' architects have to consider appropriate technology, tools, data structure, operating system, database (if required), user interfaces, programming languages, and algorithms for computing solutions optimally among others.

Software requirements, architecture, design, construction, testing, methods and models, quality assurance, and security are discussed in detail in other chapters as independent KAs.

The Computing Foundations KA focuses on explaining the key computer science concepts a software engineer has to know well to architect, design, construct, deploy and maintain useful, high-quality software subsystems.

2. Computer Architecture and Organization [6*, C6]

Computer architecture refers to the components of a computer system designed for specific purposes. Computer organization explains how the units within the system connect and interact to achieve those purposes.

System architects must analyze the application for which the computer system is to be designed or developed; identify the critical components, including I/O devices required (along with throughput), types and quantum of memory, processing power, and coprocessors required; and choose or design appropriate computer architecture and organization. Contingencies should be built in for the resources required.

This content area discusses various computer architectures and organizations a system or software architect needs to know.

2.1. Computer Architecture [8*, C1.1]

Architecture describes what the computer or system does, and its components, such as memory, data storage devices, graphics, and the computers or processor's computing power. A computing system typically has memory, I/O devices and a central processing unit (CPU).

These components are connected through physical signal lines called a *bus*. Typically, three types of buses are used for specific purposes:

- Address bus, which addresses or accesses a specific memory location or I/O device.
- Data bus, which stores (writes) or retrieves (reads) data to and from the memory location.
- Control bus, which provides control signals from the CPU to I/O devices (read or write, enable or disable, interrupt, status, reset, etc.).

Software engineers are expected to know the details of the functioning and timing of different types of buses — first-generation, second-generation and third-generation buses; internal and external buses; serial and parallel buses; simplex, full-duplex and half-duplex buses; Mil-Std-1553Bbus, Wishbone buses, etc.

2.2. Types of Computer Architectures [8*, C4.14, C5]

2.2.1. Von Neumann Architecture [8*, C1.9]

John von Neumann designed a computer system architecture with five essential components as shown in Figure 16.2:

- Arithmetic logic unit (ALU) that performs arithmetic and logic computation.
- Memory where the program and data are loaded and executed (program and data reside in the same memory space).

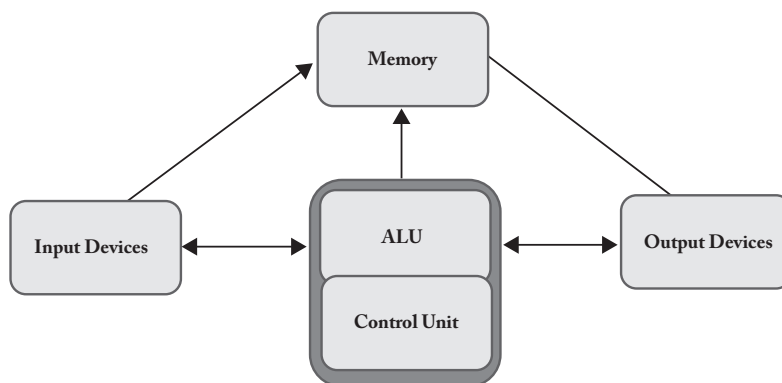


Figure 16.2. Computer Architecture

- Input devices (e.g., keyboard, mouse, serial port, hard disk) that allow the user to provide inputs and control commands.
- Output devices (e.g., monitor, printer) that transmit or communicate the computed results.
- The control unit synchronizes all devices, memory and ALU.

2.2.2. *Harvard Architecture* [20*]

The Harvard architecture provides separate memory blocks for code (program or instructions) and data. As the code and data memory blocks are different, the contents of address 0000 in the code block and the contents of address 0000 in the data block are different. The CPU reads instructions from the code addresses and reads data from the data addresses.

The system design and implementation in the original Harvard architecture were relatively complex. The modified Harvard architecture provides one memory block but partitions it into code and data sections. Data memory sections are read/write capable, and code memory sections are read-only (thus protects code from getting corrupted at runtime). I/O operations can be performed simultaneously.

2.2.3. *Instruction Set Architecture* [8*, C4.8.3]

An instruction set architecture (ISA) is an abstract model of how a CPU executes the

instruction sets defined for the system. An ISA defines registers (address, data, flags), data types, instructions specific to the computer or system, memory (internal and external) addressing schemes, and I/O handling models.

A reduced instruction set computer (RISC) architecture and a complex instruction set computer (CISC) architecture are the two primary types of ISAs.

In RISC, the instructions perform single tasks such as reading from memory or I/O, performing arithmetic or logical computation, and storing data into memory or I/O. The computer system is simple but requires more instructions to execute a task. It requires fewer clock cycles per instruction, and instruction sizes tend to be fixed. As the instruction set is small (fewer instructions), it is easier to build a compiler, and the program can be relatively large. RISC architectures are typically designed for general-purpose processors.

The instructions are relatively more powerful in CISC and can perform multiple tasks such as reading data from memory + performing arithmetic operation + storing the result in memory. Here, fewer instructions are required to perform a task, but the instructions take more clock cycles to complete. Instruction sizes vary widely depending on operations with registers, memory and I/O. Programs are relatively small. CISCs are typically designed for specific purposes such as digital signal processing (DSP) and graphics.

2.2.4. *Flynn's Architecture or Taxonomy* [8*, C9.3]

The computing architectures described above consider a single computer at a time. Michael J. Flynn proposed concurrent computer architectures, where multiple instruction streams and multiple data streams are used in the system. Software engineers need to know the different types of Flynn's architecture, with examples, including the following:

- Single instruction, single data stream (SISD) architecture.
- Single instruction, multiple data stream (SIMD) architecture.
- Multiple instruction, single data stream (MISD) architecture.
- Multiple instruction, multiple data stream (MIMD) architecture.

Variants of these architectures include array processing, parallel processing, and associate processing; processing single program multiple data streams, and multiple program multiple data streams. Software engineers are expected to know the differences among these architectures, along with case studies, so that they can choose the right architecture to solve the problem at hand.

2.2.5. *System Architecture* [6*, C6]

System architecture is the overall system design, considering hardware architecture, software architecture, modules, interfaces, data management, and communication among modules. Distributed computing has become affordable with the development of efficient, high-end, high-performance servers, storage, network devices, software, and tools. Several reference designs or architectures are available for any given application.

Typical system architectures include the following:

- Integrated system architecture: Computing, I/O, data and networking are tightly coupled and available in

one box. This architecture is typically used in solutions designed for specific applications.

- Distributed system architecture: Computing and storage are located in separate but networked boxes. This architecture supports scaling, provides centralized or isolated data storage, and shares computation load.
- Pooled system architecture: Several computing, storage and network resources are available in pools and provided depending on demand. This architecture provides for efficient use of shared resources.
- Converged system architecture: As the name implies, this is the convergence of distributed and pooled architectures. This architecture supports agility and scalability.

Software engineers are also expected to know and be able to apply various other architectures, including .NET Framework architecture, Unix architecture, and virtual machine architecture.

2.3. *Microarchitecture or Computer Organization* [8*, C4]

Microarchitecture or computer organization explains how the ISA of a computer is implemented and how different components in the system function and interact with one another to produce the desired outcome.

System architects and engineers must know the various components used in the system along with how they function. Some of these components are discussed below.

2.3.1. *Arithmetic Logic Unit* [8*, C1.2]

The ALU performs all arithmetic computations and logical operations. The CPU typically has an ALU, processor, memory, and control unit. High-end CPUs may also have other functionality-specific processing units, such as a floating-point unit (FPU), to perform computations involving floating point or real numbers (fractions). ALUs have registers

that are high-speed memory and internal to the ALU. The ALU executes the processor instruction sets. All operations are typically carried out on the registers.

Various schemes may be implemented to improve the performance of the ALU, including pipeline processing and parallel processing. The latest CPUs provide multiple cores and multiple threads that help achieve maximum throughput. Software engineers are expected to know the differences between multiple cores and multiple threads, along with specific cases illustrating the best use of these.

Specific-purpose coprocessors and associate processors are used with main processors to support faster processing.

2.3.2. *Memory Unit* [8*, C6]

Memory units are used to store data or information, which is accessed by the CPU. The total amount of memory a computer can have is derived from the maximum number of address lines supported by the CPU. Different types of memory used in the system include read-only memory (ROM), and read-write memory or random access memory (RAM).

Software engineers working on performance-critical applications are expected to know the differences among various types of memory, including static RAM (SRAM), dynamic RAM (DRAM), asynchronous DRAM (ADRAM), synchronous DRAM (SDRAM), double-data-rate SDRAM (DDR SDRAM), rambus DRAM (RDRAM), and cache DRAM (CDRAM), along with pros, cons and use cases of each.

2.3.3. *Input/Output Devices* [8*, C7]

As the names imply, input devices are those that provide inputs to the computer system, and output devices are those that deliver computer systems' output to the user. While some devices are input only (keyboard, mouse, microphone, etc.) or output only (printer, monitor, speakers, etc.), a few devices serve as both input and output devices (e.g., touch screens, hard disks, USB drives).

Software engineers are expected to understand the interface of the I/O devices with the system, whether they are memory-mapped I/O or I/O-mapped I/O devices, and device drivers required for the users or applications to interact with the devices through the operating system.

2.3.4. *Control Unit* [8*, C4.2]

The control unit synchronizes multiple components in the computer system. Typically, control units are part of the CPU. They interpret instructions and coordinate data movement among different components (memory, I/O devices and ALU). Control units are also used to enable or disable components or devices and reset devices.

Software engineers are expected to be aware of the different types of control units, including hardware control units and micro programmable control units (single-level and two-level control stores), along with the benefits and challenges of each.

3. **Data Structures and Algorithms** [8*, C2] [18*, C10 Part V]

Data structures are fundamental to computer science and software engineering. Every program uses data — receives input (data), performs specific functions on the data and produces output. Data structures is about representing different types of data effectively, performing various operations on the data proficiently, and storing and retrieving data efficiently. Software engineers must internalize data structures, the selection of data structures, and operations on them specific to applications.

In this chapter, different types of data structures and various operations on them are discussed.

3.1. *Types of Data Structures* [18*, C10], [5*, C2.1 – 2.6]

Data type is an attribute of data. Various data types are identified and defined based on different characteristics of data, the need for grouping data items and various operations

performed on data. Data structures are grouped primarily based on the physical and logical ordering of data items.

Primarily, data is grouped into three types: basic, composite or compound, and abstract.

Basic or primitive data types include character, integer, float or real, Boolean, and pointer data.

Compound data types are made of multiple basic or primitive, or even multiple compound data types. Some of the compound data types include sets, graphs, records and partitions.

An abstract data type (ADT) is defined by its behavior (semantics) from the user's perspective, specifically from the point of possible values and operations.

Composite or compound data types are further grouped under linear, and hierarchical or nonlinear data types.

Linear data types include one-dimensional and multidimensional arrays, strings, linked lists (singly linked lists, doubly linked lists, circular lists), stacks, queues, and hash tables.

Hierarchical or nonlinear data types include trees, binary trees, n-array trees, B trees, B+ trees, weighted balanced trees, red-black trees, heaps, binary heaps and graphs.

In the current era of free text queries or natural language processing, software engineers may need to understand strings and various operations on strings, and to be able to analyze skip lists.

Software engineers must understand the nuances of various types of data and their sizes in memory (short integer, integer, long integer, long long integer, signed and unsigned integer, float, double, long double, double byte character set (DBCS), Boolean, etc.), along with how various data types are represented and stored in memory and how various operations are performed on them. Sets, graphs, and trees are discussed in more detail in the Mathematical Foundations KA.

3.2. *Operations on Data Structures*

[5*, C2.1 - 2.6]

Basic operations performed on data structures include create, read, update and delete (CRUD).

Compound data types also require various ways of traversing data sets to identify specific data items before performing the operation.

It is important to ensure that any insertion or deletion of items in a data set or database does not alter the data set or database in a way that violates any policy under which the database was designed and built.

Additional operations performed on data structures include sorting the data items in a specific order, searching and locating a data item, and merging two or more data sets into one set without disturbing the policy on which the data set is built. Searching and sorting algorithms are discussed in the next section.

Different data structures are created to suit specific applications, such as stacks, queues, trees, and graphs. Software engineers are encouraged to learn the traversals through nonlinear data structures, which include different tree parsers (pre-order, in-order, and post-order tree traversals), CRUD operations on trees, tree balancing, binary search trees (BSTs), AVL trees, and red-black trees, and to learn tree search algorithms (depth first, breadth first, shortest paths, etc.). Some of these are discussed in the Mathematical Foundations KA.

3.3. *Algorithms and Attributes of Algorithms*

[18*, C26, C27]

All software implements logic to perform the required function. That logic or algorithm to perform a specific task has to be designed or chosen with consideration for system performance, security, portability, maintainability, scalability and simplicity, among other concerns.

The complexity of an algorithm is determined by measuring the computational resources (computing power and space) consumed by that algorithm for a given set of data.

A thorough understanding of data structures is vital for analyzing and designing good algorithms. Refer to the "Data Structures and Organization" content area for more details.

The attributes of algorithms are many and include functionality, correctness, robustness,

Asymptotic Notations	Description
Big O	Big O notation provides the upper bound of operations (worst-case scenario) for a function $f(n)$.
little-o	Little o notations are used to depict scenarios where the upper bound is not tight.
Big Omega (Ω)	Big Ω notations are used to depict lower bounds (best-case scenarios) for a function $f(n)$.
little-omega (ω)	Little omega (ω) notations are used to depict loosely bound best-case scenarios of an algorithm.
Theta (Θ)	Theta notation bounds the function from above and below (provides average-case complexity of an algorithm).

Table 16.1. Asymptotic Notations of Algorithms

modularity, maintainability, programmer-friendliness (ease of integration into the project and ease of use), user-friendliness (i.e., how easily it is understood by people), need for programmer time, simplicity, and extensibility.

A commonly emphasized attribute of algorithms is “performance” or “efficiency.”

The parameters that matter for an algorithm’s resource consumption include, but are not limited to:

1. Hardware.
2. Software.
3. Algorithm selection and design for a specific problem.
4. Effective implementation.

3.4. Algorithm Complexity [5*, S1, S3, S4, S5, S6, S7, S11, S12]

The complexity of an algorithm is a measure of the resources it consumes (computing power or memory) for a specific problem and given data set.

Choosing the right data structures and operations on data structures and ensuring optimal implementation of the algorithm also effect the algorithm’s complexity.

3.5. Measurement of Complexity [5*, S1.1, S3, S4, S5, S6, S11.1, S12.1]

Often, the complexity of an algorithm is denoted by the resources consumed in the

worst-case scenario. The complexity of algorithms is typically measured by asymptotic notations for best-case, worst-case and average-case scenarios in terms of resource consumption for a given data set.

Popular asymptotic notations for algorithms are listed in Table 16.1.

Learning the computation of the listed notations for different sets of input data (e.g., sorted, unsorted, and sorted in reverse order) is important.

The complexity of an algorithm can be constant, linear, quadratic, cubic, exponential or logarithmic. These complexities are described in Table 16.2. Typically, constants are not considered when computing the efficiency of an algorithm.

3.6. Designing Algorithms [18*, Part IV, Part VI]

The software engineer must consider the specific application’s purpose and the performance requirements in order to select an appropriate algorithm. In addition, the software engineer must consider linear programming versus parallel programming and single- versus multi-threading.

The efficiency of an algorithm is measured by the resources it consumes, primarily computing time and memory.

A software engineer has to know a few standard algorithms and relevant concepts, including the following:

Complexity	Notation	Description
Constant	$O(1)$	Regardless of the data size, the algorithm takes a constant number of steps to perform the operation.
Linear	$O(n)$	The number of operations is linearly proportional (steps are a constant multiple of the data set size n).
Quadratic	$O(n^2)$	The algorithm takes the order of n^2 steps for performing the operation on the data set of size n .
Cubic	$O(n^3)$	The algorithm takes the order of n^3 steps for performing the operation on a data set size of n .
Exponential	$O(n^k)$ $O(2^n)$ $O(n!)$	The algorithm has an order of exponential dependability for performing the operation on a data set of size n .
Logarithmic	$O(\log(n))$ $O(N^{\log(n)})$	The algorithm takes the order of $\log(n)$ steps (base of log is typically 2).

Table 16.2. List of Algorithmic Complexities

- Common types of algorithms: Brute force algorithm, Recursive algorithm, Divide & Conquer algorithm, Dynamic programming algorithms, Greedy algorithm, Backtracking algorithms, Randomized algorithms.
 - Randomized approximation algorithms, randomized rounding, approximation algorithms, P and NP complexity class algorithms, Cook's theorem, reductions and completeness algorithms.
 - Multiple comparison operations performed simultaneously in a network model of computation. Popular sorting network algorithms include comparison networks, zero-one principle, merging network and bitonic sorter.
 - Optimized algorithms for performing several operations on a matrix, such as matrix multiplication, transposition, matrix inversion, median, and finding determinants.
 - Cryptographic complexity and algorithms: secret key (symmetric) encryption algorithms, public key (asymmetric) encryption algorithms and hash functions.
 - One-way functions, class UP, space complexity, deterministic and nondeterministic space complexity classes, the reachability method, and Savitch's theorem.
 - Graph representations, graph algorithms, breadth-first and depth-first search, topological sort, minimum spanning tree, Kruskal and Prim algorithms, and single-source shortest paths (Bellman-Ford and Dijkstra algorithms).
 - Complexity of randomized computation, interactive proofs, complexity of counting, Boolean circuit complexity.
- Of particular importance in many software systems are algorithms for sorting and searching, these are discussed in more detail.

3.7. Sorting Techniques [18*, C6-C9]

Sorting is the process of arranging data items in a specific order.

Popular sorting algorithms include Linear sort, Bubble sort, Quick sort, Merge sort, Radix sort, Heap sort, Bucket sort, Pigeonhole sort, Bitonic sort, Tree sort, Cartesian Tree sort, 3-Way Quick sort, 3-Way Merge sort, and Sorting Singly / Doubly linked lists.

Each sorting algorithm has its benefits and shortfalls. Selection of an appropriate algorithm depends on the size of input data, the type of data (linear or nonlinear), and the type of data set (completely unsorted, partially sorted, etc.). The algorithms are implemented in both iterative and recursive methods.

Typically, iterative methods are better than recursive methods for CPU performance and memory. However, recursion provides easy methods for solving specific problems, such as tree operations. If adequate computing power and memory are available, the difference between recursive and iterative implementation methods is negligible.

In the case of applications where certain sorting algorithms work best, software engineers should learn and accommodate any preconditions and complexities (demand on memory and computing power) involved in using them.

3.8. Searching Techniques [5*, C6]

Searching is a process of finding specific data items or records in a set of data items or a database.

Search algorithms are primarily categorized into sequential search (data set is traversed sequentially until the end of the data set) and interval search (the search moves efficiently through a sorted list, balanced tree, etc.), based on how data sets are organized.

Depending on the type of the data item and the size of the data set, various search techniques are used to find the desired data item. Popular search algorithms include linear, binary, jump, interpolation, exponential, Fibonacci, sub-list (search a linked list in another list), logarithmic, tree and hashing.

3.9. Hashing [18*, C11.2]

Hashing is one of the very important and popular technique in which data of arbitrary size (key values) are converted into values of fixed size called *hash values*, which index into a hash table so the data records can be located easily. The function used for that purpose is called a hash function, and the values returned are called hash values, hash codes, digests, or hash keys.

Different properties of hash functions, such as uniformity, efficiency, universality, applicability, deterministic, defined or variable range, data normalization, testing, and measurement,

must be understood and considered when designing or choosing a hash function.

Various types of hash functions are designed for different types of key values, applications, and database sizes. Hash function types include trivial hash function, division method, mid-square method, digit folding method, multiplicative hashing, double hashing, open and closed hashing, rehashing, extendible hashing, and cryptographic and noncryptographic hash functions.

Software engineers are expected to learn, implement and be able to compare different types of hashing algorithms, various collision resolution techniques, linear probing, quadratic probing, separate chaining, and open addressing.

4. Programming Fundamentals and Languages [4*, C6]

Computer programs are sequential steps or instructions that work on provided inputs and generate desired or specific outputs.

Software engineers must carefully consider various aspects before selecting a programming language to solve a specific problem.

4.1. Programming Language Types [8*, C8.4.4]

Depending on the hardware, operating system, and application various types of programming languages are developed and used. Basic types of programming languages include microprogramming, machine languages, assembly programming and high-level programming.

Microprogramming is executed within the microcontroller or microprocessor chips to execute the assembly language instructions.

Assembly language programs use the mnemonic specified by the microcontroller or microprocessor. Typically, the microcontrollers or microprocessors are designed to address specific applications (DSP processors, graphics chips, I/O controllers, mathematical coprocessors, generic processors, etc.).

High-level languages enable programs to be written in instructions similar to English,

which makes it easy for the developer and maintainer to write and maintain the programs. Various types of high-level programming languages include the following:

- Functional programming languages.
- Procedural programming languages.
- Object-oriented programming languages.
- Scripting languages.
- Logic programming languages.

A programming language can support more than one programming paradigms. Software engineers need to study multiple programming languages to choose the right one for a specific application.

Many programming languages, such as C, C++ and Java, use compilers to build executables, whereas other programming languages, such as JavaScript, Ruby and Python, use interpreters.

4.2. *Programming Syntax, Semantics, Type Systems* [8*, C8.4.4]

The syntax of a programming language is its grammar — the various constructs the programming language uses. A compiler or interpreter checks the syntax of all declarations, statements (algorithmic statements, conditional or logical statements, control statements, loops, special language-specific statements, macros, etc.), and functions or procedures, and creates notifications of any errors.

Semantics refers to the meaning or interpretation of the statement. The meaning could vary at runtime, depending on runtime values.

A type system assigns a type to a data item or to constructs of a program, such as variables, expressions and functions. In static typing, the type is fixed; it is defined during program creation and checked at compilation time. Languages such as C, C++ and Java support static typing. In dynamic typing, the type of a variable can change at runtime depending on the context and hence is checked at runtime. Dynamic typing languages include Python, Perl, PHP and Ruby. Dynamic typing is also called polymorphic typing.

Software engineers are expected to know how high-level programming languages are translated into machine languages, to be familiar with the various types of compilers, and to know the differences among compilers, interpreters, cross-compilers, assemblers and cross-assemblers. Software engineers are encouraged to learn about compiler phases, including preprocessing, lexical analysis, syntax analysis, intermediate code generation, optimization, code generator, linkers, loaders and debuggers.

Tokens, grammars, syntax trees, parse trees and weights to various operators (precedence) in arithmetic and logical equations are important to analyze and understand.

4.3. *Subprograms and Coroutines* [4*, C6.3]

Subprograms or functions are programs or building blocks that perform specific (part) functions in the scope of a complete project. Subprograms provide for breaking the larger program into smaller modules. The modules are typically sections of code that are used multiple times in multiple places. The subprograms reduce memory space, improve readability and maintainability of the program, and execute parts of the program with different values at different places and times.

The subprograms have an entry point and typically have multiple input parameters on which the subprogram acts and produces output. The scope of input parameters is local to the subprogram. Subprograms that return value by their name (which can be used as a variable in a statement) are called *functions*, and subprograms designed not to return any value are called *procedures*.

By default, the scope of subprogram parameters is dynamic and local to the subprogram. However, if the subprograms have to remember their history or previous values, they have to be declared static or as specified in the chosen programming language.

Different programming languages support one or more types of parameters' passing, including pass-by-value, pass-by-reference, pass-by-name, pass-by-result and

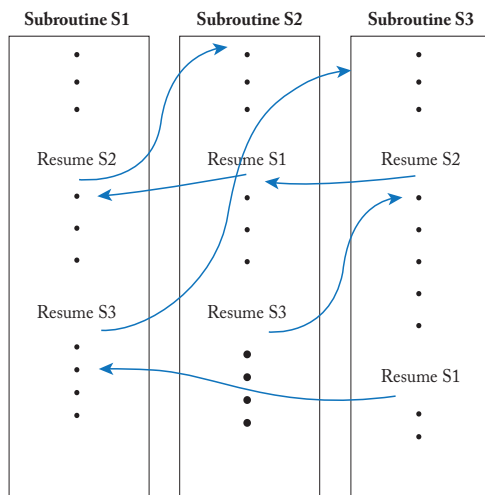


Figure 16.3. Example of Coroutine

pass-by-result-value. Software engineers should know the differences among these types and use them appropriately.

Many high-end languages support the nesting of subroutines and recursions, where a subroutine calls itself. Different types of recursions include cyclic or direct recursion (subroutine calls itself) and acyclic or indirect recursion (subroutine A calls subroutine B, which in turn calls subroutine A). It is important to establish the exit criteria in recursive subprograms.

Software engineers are encouraged to understand, using case studies, how the subprogram return address and parameters are stored in memory (runtime stack), how they are used in the subprogram and for returning to the called subprogram, and the scope of variables (global and local).

A subprogram with multiple entry points, where the previous exit point is remembered for resumption at a later point, is called a **Coroutine**. A Coroutine call is typically called a *resume call*. The first resume call enters the subroutine from the beginning, and subsequent resume calls enter the subroutine at the point where it was exited last.

High-end languages that support Coroutines include C++20, C#, Java, JavaScript, Kotlin, Perl, .NET Framework, Python, Ruby and many assembly languages.

Software engineers are encouraged to understand specific applications where coroutines are useful and to use the coroutines. It is an interesting exercise to implement coroutines in C, as C does not support coroutines natively.

Figure 16.3 depicts the functioning or control flow of coroutines.

4.4. Object-Oriented Programming [4*, C6.5]

As the name suggests, object-oriented programming languages are based on objects. The objects typically have both data and functions that operate on that data. The data of an object is typically called the object's attributes or properties, and the code or functions that work on the attributes are called operations externally (by the client or user) and called methods internally (referring to how the operation is implemented by the developer).

A Class is a programmer-defined prototype that defines the attributes and methods. Objects are actual instances of a Class. There could be multiple Objects of a Class with varied characteristics. For example, a Class can be defined by the characteristics and operations of a vehicle, whereas objects are instances of the class vehicle such as car, bus or truck.

The objects interact with one another using the methods or operations.

Important characteristics of object-oriented programming (OOP) are Abstraction, Encapsulation, Inheritance and Polymorphism.

Abstraction is a property that exposes only required or relevant information and functionality to the user, hiding the details and nonessentials. Thus, the implementation is hidden from the user of the superclass.

One of the key benefits of encapsulation is the ability to hide or protect data from unauthorized users. The software engineer can give different levels of protection to data and methods by declaring them private (local to class) or public (available to other classes). This also protects data from corruption, either intentional or accidental.

Inheritance is an important feature of OOP, where a subclass or derived class inherits the properties of a superclass or base class. Primary inheritance modes include public, protected and private modes.

Polymorphism is another key feature of OOP. Polymorphism is a provision of providing a single interface to entities of different types. For example, shape could be a base class with draw as a method, and objects could be a circle, triangle or rectangle. The implementation of method draw, though the name is the same, differs for a circle, triangle and rectangle. Polymorphism has two types:

- *Static or compile-time polymorphism:* The methods (functions) or operators are overloaded and resolved during compile time. Example: The methods, though they have the same name, will have different types or numbers of parameters.
- *Dynamic or runtime polymorphism:* The overloaded method to be executed is resolved at runtime. Example: When both base class and derived class have the same method, the base class method is said to be overridden.

Popular OOP languages include C++, C#, Cobol 2002, Java, Python, Lisp, Perl, Object Pascal, Ruby and Smalltalk.

It's important to recognize that using OOP requires a different mindset than using traditional, procedural, or structured programming does.

4.5. *Distributed Programming and Parallel Programming* [4*, C6.6]

In a distributed computer system, multiple parts of the software are run on multiple computers, connected through computer networks, to achieve a common goal. Writing such programs is called *distributed programming*.

Parallel programming is a type of computing in which different parts of the program are run in parallel to achieve the same objective or goal. Table 16.3 compares distributed

and parallel programming. High Performance Computing (HPC) aims to speed-up the execution of software, both distributed programming and parallel programming are ways to do this and is increasingly used together in hybrid software.

4.6. *Debugging* [6*, C2.2.2]

Programs, when written, are expected to function properly and generate the expected output. However, programmers often face three types of errors — syntax errors, runtime errors, and logical errors — at different stages of software development.

Syntax errors are deviations from the standard format specified by programming languages. These are explicitly identified by compilers and are easy to fix.

Runtime errors surface when a program runs into an unexpected condition or situation such as dividing by zero, memory overflow, or addressing a wrong or unauthorized memory location or device, or when a program tries to perform an illegitimate or unauthorized operation or tries to access a library, for example. The programs must be thoroughly tested for various types of inputs (valid data sets, invalid data sets and boundary value data sets) and conditions to identify these errors. Once identified, runtime errors are easy to fix.

Logical errors are slipups in implementing the logic to achieve the desired output. These errors must be traced and resolved with various data for each functionality. Several sophisticated high-end debuggers help trace each variable or data item and support setting various types of break points.

4.7. *Standards and Guidelines* [3*, C28.5, C31.5]

As the computing system or application becomes bigger and complex, more programmers are involved. Their individual programming styles affect the project schedules and make system integration difficult, so systems become defect-prone, and maintenance and enhancement become challenging.

Parameters	Distributed Programming	Parallel Programming
Functionality	A task is shared and executed by multiple computers that are networked.	Two or more processors on a computer share and execute the task in parallel.
Computers	Multiple computers in different locations but networked.	Two computer with one or more processors or cores.
Memory	Each computer has its own memory.	Computers can have shared or distributed memory.
Communication	Computers communicate through networks.	Processes communicate through a bus or inter-process communication (IPC) methods.
Benefits	Failure of one computer does not affect the functioning of the task, as it is transferred to another computer. Provides scalability and reliability for end users.	As multiple processes run in parallel, generally the performance increases. Failure of one processor does not affect the performance of other processors or cores
Disadvantages	Having multiple systems could become expensive; the cost must be weighed against customers' need for application uptime. Network delays could affect the overall functioning of the task. Designing an efficient distributed computing system is relatively difficult.	Using multiple processors or cores could be expensive. Dependency of one process on another process could introduce latency.
Example Applications	Telephone and cellular networks, internet, World Wide Web networks, distributed database management systems, network file systems, grid computing, cloud computing.	2D and 3D simulations and rendering in computer graphics, scientific computing.
Example Programming Languages, libraries engines, frameworks	Golang, Elixir, Scala, Fortran, C and C++.	Apache Hadoop, Apache Spark, Apache Flink, Apache Beam, CUDA, OpenCL, OpenHMPP, MPP, OpenMP for C, C++ and Fortran.

Table 16.3. Comparison of Distributed and Parallel Programming

An estimated 82% of vulnerabilities are caused by clashes between programming styles.[§]

Hence, quality-conscious companies often have defined tools, standards and guidelines, which set rules and recommendations for their programmers and testers to follow.

When software teams follow appropriate coding standards, they create readable, cleaner, portable, reusable, modular,

easily maintainable, less defect-prone software code, and project schedules become more predictable. The following practices can help organizations implement such standards successfully:

- Carefully choose the coding standards and guidelines that suit the application or system being developed.
- Consider open standards created by

§ <https://www.ptsecurity.com/ww-en/analytics/web-vulnerabilities-2020/>

community participation, such as Software Engineering Institute (SEI) Computer Emergency Response Team (CERT), as well as closed standards created by working groups such as the Motor Industry Software Reliability Association (MISRA).

- Educate programmers to follow adopted standards and guidelines.
- Use tools and periodic reviews to ensure adopted standards and guidelines are followed.
- Review and revise standards and guidelines from time to time, learning from project execution.

SC 22 is a subcommittee of the Joint Technical Committee ISO/IEC JTC 1 of the International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC) for defining standards for programming languages, their environments and system software interfaces (ISO/IEC JTC 1/SC 22). Software engineers are recommended to refer these standards as well.

5. Operating Systems [19*]

An operating system (OS) is software that manages the computer's hardware and provides a platform for software applications. Software engineers need a good general understanding of OSs and OS objectives, services, and functions.

Different types of OSs have been designed over time to support various types of systems or applications, including batch processing, multiprogramming, time-sharing, and dual-mode operation — for protecting I/O, memory, CPU, kernels and micro-kernels.

To choose an appropriate OS, software engineers have to analyze different types of operating systems, such as single-user, single-tasking, multiuser, multitasking and multi-threading OSs; real-time OS (RTOS); network OS; and distributed OS. For small systems, an operating system may not be required. It is important to study examples of each type and compare their benefits and limitations.

Software engineers need to understand operating systems' basic structure, system architecture types, design approaches, the architecture of distributed OS and issues in distributed OS.

An operating system typically has four major components: processor management, memory management, device management and information management.

5.1. Processor Management [19*, C2, C8]

Software engineers must understand the concepts of processor, process and address space. They must understand booting, processes, cores, threads, user and kernel threads, fork and exec, synchronization, and hardware support for locking. They should compare and contrast various CPU scheduling concepts, scheduling algorithms, algorithm evaluations, multiple processor scheduling and real-time scheduling, concurrent programming, deadlocks, critical regions, conditional critical regions, and monitors.

Communication among different processes is important in multitasking, multiuser OSs. A software engineer must have a deep understanding of inter-process communication (IPC), and types of IPCs, including messages, pipes, shared memory, semaphores, modularization and process synchronization.

Various types of locks are used to ensure proper synchronization of data among processes, including semaphores, binary semaphores, counting semaphores and mutex locks. Deep understanding of common challenges of IPCs, deadlocks, deadlock scenarios, and deadlock characterization; prevention, avoidance, detection and recovery of deadlocks; and precedence graphs is critical and to be internalized with the help of case studies.

Software engineers are required to study, with examples, concurrent languages, processes and scheduling, job and process concepts, and various types of scheduling: CPU-I/O interleaving, non-preemption, context switching, and scheduling algorithms (first come, first served (FCFS), shortest job first (SJF), shortest remaining time first

(SRTF), priority scheduling, round robin and combined schemes).

5.2. *Memory Management* [19*, C3]

A software engineer needs a very good understanding of how memory is managed in the system and of the different types of memory and relevant concepts — physical memory, virtual memory, secondary memory, memory hierarchy, linking and memory allocation.

Engineers must understand memory fragmentation (both external fragmentation, internal fragmentation), and various memory management concepts, including units, paging, page tables, segmentation, paged segmentation, virtual memory management, demand paging, page replacement, thrashing and swapping.

Memory is allocated to processes in different ways — for example, through contiguous allocation, noncontiguous allocation, dynamic partitioned memory allocation, static-swapping and overlays.

An understanding of logical addresses, partitions, static versus dynamic memory allocation, free space management, and defragmentation of memory blocks is also important.

As the physical memory available is always limited, various memory page replacement strategies are designed and implemented. These strategies include first-in-first-out (FIFO), not-recently-used (NRU), least recently used (LRU), most recently used (MRU), least frequently used (LFU), most frequently used (MFU), longest distance first (LDF), second chance, and aging among others.

5.3. *Device Management* [19*, C5]

A software engineer must have good knowledge of different types of I/O devices — memory-mapped and I/O-mapped devices, block and character devices, and buffering devices. Engineers should compare and contrast polled, interrupt-driven and direct memory access (DMA) I/O devices, and blocking versus non-blocking I/O devices.

Device drivers are software programs that provide an interface between hardware and applications. Software engineers should understand device drivers, the various types of device drivers, device driver tables, device driver functions, and interfaces for various types of hardware devices, as well as hardware and software interrupts and interfaces by interrupts and polling.

Software engineers should also understand that issues with caching, scheduling, spooling and performance can arise for shared devices in multiuser, multitasking OSs and device a mechanism for resolving them.

5.4. *Information Management* [19*, C4]

Software engineers need to understand the following:

- The concept of a process, a system programmer's view of processes, an operating system's view of processes, and operating system services for process management
- File system management, storage management, file attributes, directory structure, file system structure, mass storage structure, I/O systems, protection and security
- User and operating system views of the file system and various types of file systems — simple file system, symbolic file system, logical file system and physical file system

Engineers should be familiar with various operations including access control lists (ACLs), access matrix, access control, access control verification, capabilities allocation strategy, I/O initiators, device strategy, device handlers, disk scheduling, disk space management, existence and concurrency control, schemes and combined schemes, authentication schemes, directory namespace, hierarchies, directed acyclic graph (DAGs), hard and soft links.

5.5. *Network Management* [4*, C4.1]

Network management is the process of administering and managing various types of networks. This content area includes network

management concepts, distributed objects, distributed file systems, and network architecture, design, issues and resolutions.

A network manager will need detailed knowledge of physical and logical time, as well as internal and external synchronization protocols in network management such as Cristian's algorithm, Berkeley's algorithm, the Network Time Protocol, Lamport's logical clock, Vector clocks, Casual ordering of messages, and global state.

Other important topics include distributed computation, termination detection, distributed mutual exclusion and election, simple and multicast-based mutual exclusion algorithms; Centralized, Ring based, Ricart Agrawala's algorithm, Maekawa's algorithm, Election algorithms, Bully's algorithm and multicast communication.

In addition, software engineers should understand important principles include hardware security, external security, operational security, password protection, access control, security kernels, and the layered approach.

6. Database Management

A database is a collection of related data elements, collected specifically for use by one or more applications and stored in an organized format for easy and quick access, using one or more key values. The data items or elements are stored in one or more databases or files, and the relationship among them is established using a database schema.

Basic operations performed on the database include creating the database and its elements (table, index, views, functions, procedures, etc.), deleting or dropping items from the database, modifying contents and structure of the database, and data retrieval, comment, and rename actions.

Different types of databases include relational databases, not only structured query language (NoSQL) databases, columnar databases, object-oriented databases, key-value databases, document databases, hierarchical databases, graph databases, time series databases, and network databases. Understanding

what type of database works best for specific applications and analyzing the definition, structure, specific pros and cons of each type of database; what along with examples helps software engineers choose the right type of database for a given application.

When selecting a database, software engineer should evaluate data models, storage models, types of databases, key values, graphs, column family, volume of data, consistent data access time, and the number of users or applications accessing the database (traffic), etc.

The learners and users of the database system need to create two roles (database user and database architect), review several case studies of increasing complexity, create multiple databases, and analyze the information. This process significantly helps one to understand and internalize the database design and management.

6.1. Schema

[22*, C2.1.4]

A database schema is a structure or record of data items, defined in one or more database tables, and the relationships between them. The schema may also contain formulae to check the integrity of data items, relationships, indexes, functions or procedures and views.

While a physical schema explains how the database is designed at physical level (files), the logical schema describes how different data items are defined in one or more tables and interconnected.

Different types of schemata used in the industry include star, snowflake and fact constellation schemata. Different types of keys used in schemata include Primary Key, Secondary / Alternate Key, Foreign Key, Composite Key, Surrogate Key and Candidate Key.

Parameters that influence the definition and use of schemata include overlap preservation, extended overlap preservation, normalization and minimality.

6.2. Data Models and Storage Models

[22*, C2.3]

A data model specifies the logical aspects of data structure in a data store, and a storage

model specifies the physical aspects of data structure in a data store. It is difficult to achieve both data consistency and high availability in a database.

The two primary data models used to distinguish databases are the following:

- The ACID (atomicity, consistency, isolation, durability) model provides for high data consistency. ACID-compliant databases are ideal for a finance-intensive application.
- The BASE (basically available, soft state, eventual consistency) model provides flexible methods to process data, which suits NoSQL database types.

Types of storage models include the following:

- DAS (direct access storage): Storage devices are physically or directly connected to the computer that processes the data.
- NAS (network access storage): Data is stored in a network and accessed by multiple computers or applications.
- SAN (storage area network): Data is stored in multiple servers and efficiently provided to users through a computer network.

6.3. Database Management Systems [22*, C1.3]

Database management systems (DBMSs) are software systems that provide the necessary tools for maintaining data optimally, retrieving stored information effectively, protecting and securing stored data, and managing access for users of different levels of authority.

Typical DBMSs include:

- *A database engine:* This is the core of a DBMS. The database engine manages efficient storing and retrieving of data. Users with privileges can access the database engine.
- *A database manager:* This program or set of programs performs all DBMS functionality in a database (creating, purging,

backing up, retrieving, maintaining, cloning and deleting data). It is also responsible for maintaining the DBMS with patches and updates.

- *A runtime database manager (RDM):* The RDM checks for user authentication and privileges before any operation is performed, provides access to a context-based database, provides concurrent access to the database by multiple users, and ensures data integrity.
- *Database languages:* These help in storing, retrieving, modifying and retrieving data, controlling user access (privileges), specifying schemata and views, and performing various operations. Popular database languages include data definition language (DDL), database access language (DAL), data manipulation language (DML), Transaction Control Language (TCL), and data control languages (DCL),
- *A query processor:* This basic and key component of DBMS provides an effective, rich and English-like interface for users to access the database and perform various functions or operations.
- *Reporting:* Reporting applies specified filters, extracts requested data and records from one or more database tables, and presents information as specified.

Several free and open-source database management systems are available.

6.4. Relational Database Management Systems and Normalization [22*, C4]

Conventional file system-based databases suffered from data redundancy, data inconsistency, data access challenges, unauthorized access, lack of concurrent access, among other issues.

A relational database management system (RDBMS) stores data in tables and, unlike in a DBMS, its data tables relate to one another, multiple data items can be accessed simultaneously, a large amount of data is handled, multiple users can access data concurrently, data redundancy is significantly reduced, and

multiple levels of data security are supported.

Computer science engineers must understand the difference between the various types of RDBMS, such as Objective RDBMS, Object Oriented RDBMS, be familiar with examples, and know the applications they suit best.

Database normalization is the process of organizing data in a database and removing data redundancy and data inconsistency from the tables. Normalization might increase the number of tables and increase the query time. If this occurs, then — depending on the application and the requirement — de-normalization is applied, where data redundancy is added for quicker data access.

Different types of database normalizations are the following:

- i. First normal form (1 NF): Removes duplication or redundancy. Each table cell has a single value (creates more entries and tables). Each row has unique values. Related data is identified with a unique key.
- ii. Second normal form (2 NF): The table should be in 1 NF; no partial dependency (creates separate tables with records referenced by multiple records or tables).
- iii. Third normal form (3 NF): The table should be in 2 NF. Transitive dependencies are removed.
- iv. Boyce-Codd normal form (BCNF/3.5 NF): The table should be in 3 NF, and X should be the super-key for any (X→Y).
- v. Fourth normal form (4 NF): The table should be in 3.5 NF and should not have a multivalued dependency.
- vi. Fifth normal form (5 NF): The table should be in 4 NF and cannot be split into any more tables without losing data.
- vii. Sixth normal form or domain/key normal form (6 NF/DKNF): The table should be in 5 NF, and every join dependency is trivial.

Most databases are typically normalized until 3 NF or BCNF. An alternative normal form, DKNF, is defined where insertion and deletion of anomalies is avoided (see [13]).

Database engineers are encouraged to understand normalization forms with

examples and case studies and to understand the challenges one would face if the database were not normalized. Although normalization is essential and provides various benefits, it also increases the number of tables and processing time.

6.5. Structured Query Language

[22*, C6, C7, C8]

Structured query language (SQL) is a standard and popular database language for creating, updating, and deleting databases and for retrieving information from databases. SQL is an inevitable part of most database management systems.

Typical SQL syntax has several language constructs or elements, including clauses, expressions, predicates, queries and statements.

All operations on a database, including creating, updating, deleting and viewing tables; performing different normalizations; purging data; and searching through the database based on various combinations of parameters or filters, can be performed using SQL.

Most databases support SQL (except NoSQL databases), and the SQL syntax and library of functions supported vary across database providers (much like programming languages — though different languages support similar features, the syntaxes vary).

Database engineers also have to decide whether to use static/embedded SQL, dynamic SQL or a combination of the two, after weighing the pros and cons of each option for the particular application. They should also know the differences between simple and complex views and use them appropriately.

SQL is standardized and adopted by the American National Standards Institute (ANSI) and ISO. The standards are revised from time to time; the first SQL standard was SQL-86, issued in 1986, and the most recent is SQL:2019.

6.6. Data Mining and Data Warehousing

[22*, C34]

Databases are designed to store transactions and retrieve them efficiently.

Data warehousing extracts data from multiple databases efficiently and stores it in a common database so data mining can be performed effectively on the compiled data. Data warehouses are typically huge, as they store historical data records.

Data mining extracts requested information from the data warehouse, applying various filters and conditions. Data mining applies pattern recognition algorithms to huge data sets to generate required reports.

The different types of warehouses include enterprise data warehouse (EDW), operational data store (ODS), and data mart (DM).

Many efficient tools are available to create data warehouses and mine data from them.

Database engineers must know different data mining techniques, including association, clustering, classification, sequential patterns and prediction, and know how to apply them for various uses and industries, such as health care, fraud detection, customer relationship management, finance and banking, anomaly detection, prediction, neural networks, statistics, and data visualization.

6.7. Database Backup and Recovery [22*, C22]

Database systems are prone to failures, and data can be corrupted. It is crucial to prevent data corruption and — if it does occur — to recognize it immediately and recover the data.

Updating the database for transactions must be carried out carefully (with commits at specific checkpoints), and must incorporate techniques such as undoing, deferred updates, immediate updates, caching or buffering, and shadow paging.

Databases must be backed up periodically to ensure data safety. Backup techniques include Full database backup, Differential backup and Transaction log backup.

7. Computer Networks and Communications [4*, C4.1], [24*, C1]

A computer network is a group of devices that are connected for sharing information. The connected devices (nodes on the network)

can be located near one another, on the same premises, or somewhere else. Networking is required for certain benefits, including certain modes of communication and information sharing; the ability to share devices such as printers, routers and video cameras; global information and data storing; security and policy enforcement; remote monitoring; shared business models; and web browsing.

As we are in the internet era, computer networking is a critical element in computing, and the practitioners of computer science engineering have to study computer networks and communication concepts, including examples and case studies. Many computing paradigms (distributed computing, grid computing, cloud computing, etc.) are based on networking principles.

It is important for software engineers to understand the following:

- Different types of computer networks.
- Layered architectures of networks.
- Open systems interconnect (OSI) layers
- Encapsulation and decapsulation.
- Application layer protocols.
- Design techniques for reliable and efficient networking.
- Internet and packet delivery.
- Wireless and mobile networks.
- Security and vulnerabilities.

7.1. Types of Computer Networks [4*, C4.1], [24*, C1.2.1]

Different types of computer networks are designed and used based on the need, such as the following:

1. Personal area network (PAN) / home network.
2. Local area network (LAN).
3. Wireless local area network (WLAN).
4. Wide area network (WAN).
5. Campus area network (CAN).
6. Metropolitan area network (MAN).
7. Storage area network (SAN).
8. System-area network (SAN).
9. Enterprise private network (EPN).
10. Virtual private network (VPN).

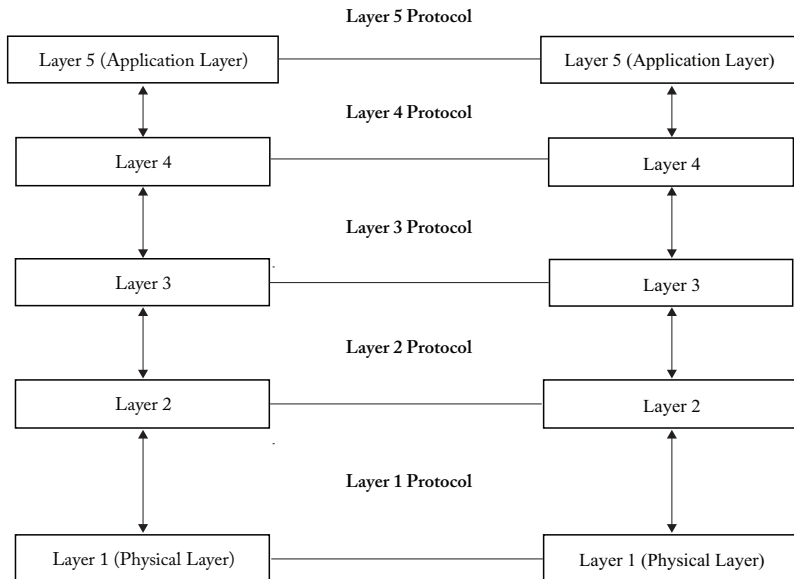


Figure 16.4. Pictorial Representation of Layered Networking

It is important to understand each of the above network type as well as examples, benefits, limitations and available solutions to circumvent challenges.

7.2. Layered Architectures of Networks [24*, C1.5]

A communication system includes hardware and software, and these components have become complex to meet complicated use scenarios and user demands. To support the implementation and maintenance of such systems, ISO has developed a layered approach, where every layer has specific functionality for processing data and transferring it from one node to another.

Each layer is independent in its functionality and provides services from the lower layer to the upper layer without providing details of how each layer's service is implemented. Each layer ("n") on a machine communicates with the same layer ("n") on the peer machine. Rules used in a conversation are called *layer-n protocol* (see Figure 16.4).

The basic elements of the layered approach are service, protocol and interface.

- **Service:** The set of actions a layer provides to the adjacent higher layer is the service.
- **Protocol:** The set of rules a layer uses to exchange information with the peer entity is called the protocol. The rules are primarily for managing both the contents and order of the messages used.
- **Interface:** The interface provides a medium for transferring the message from one layer to another layer.

Software engineers are expected to understand the essential functionalities required, various modes in which the data or information is communicated from one layer to the other, and data packet formation and interpretation at peer levels. A useful exercise is to take examples of different protocols and analyze them.

7.3. Open Systems Interconnection Model [24*, C1.5]

The Open Systems Interconnection (OSI) Model was defined by the ISO. It serves as a reference model for information exchange between applications on two systems or computers through a physical medium.

OSI proposes seven (7) layers, and each layer is assigned a specific task. Each layer independently processes the data it receives from the upper or lower layer and passes it to the lower or upper layer, as appropriate.

Engineers must understand each OSI layer, its functionality protocol, the input and output of each layer in each direction (from lower layer to upper layer and vice versa). Engineers should analyze whether all seven layers are required for all protocols and what is necessary to optimize for performance.

1. Physical Layer (Layer 1).
2. Data Link Layer (Layer 2).
3. Network Layer (Layer 3).
4. Transport Layer (Layer 4).
5. Session Layer (Layer 5).
6. Presentation Layer (Layer 6).
7. Application Layer (Layer 7).

Engineers must understand the nuances of each layer, with examples.

7.4. *Encapsulation and Decapsulation* [24*, C1.5.2]

Each layer, while sending data from the upper layer to the lower layer, inserts additional information at the beginning (header) and optionally at the end of the data packet received from the upper layer, treating the packet received from the upper layer as data. This is encapsulation. The protocol data unit (PDU), which is the data packet containing additional information from all layers, is sent to the receiving system. At the receiving end, each layer extracts its header from the PDU, deciphers the information to treat the data appropriately, and sends the remaining PDU to the upper layer.

Learning about cross-layer optimization, the principles to which it must adhere, and its applications is important. Engineers should analyze the PDU structures of each layer of OSI, the Internet protocol suite and the asynchronous transfer mode (ATM).

7.5. *Application Layer Protocols* [24*, C2]

The application layer, being the top most layer, provides services and interfaces to interact with users' application. There are two types of application layers in the OSI model: common application service element (CASE) and specific application service element (SASE). Example applications include file transfer (FTP, TFTP, NFS), remote login (Telnet, Zoho Assist, Anydesk, TeamViewer, etc), e-mail (SMTP) networking support (DNS), network management (SNMP, DHCP), devices (LPD), etc.

Software engineers practicing in a networking domain need to understand CASE and SASE application services, including example applications in each category.

7.6. *Design Techniques for Reliable and Efficient Network* [24*, C1.5]

Today's information technology-based businesses need around-the-clock, reliable, efficient and scalable networks and high-speed internet availability. Catering to varied business needs, the networks and their management has become complex as well.

It is critical to identify network requirements (both business goals and technical solutions) along with a road map (scalability). The fundamental design goals should include reliability, security, availability and manageability. Engineers should expect threats and intrusions at multiple levels and design security at multiple levels. Systems must be set up to monitor the networks for both proper functioning and malfunctioning; identify faults, vulnerabilities and hacks quickly; and fix them.

Engineers must understand and learn the nuances of designing a network while using appropriate firewalls, LAN/VLANs, subnets, quality of service (QoS), Demilitarized Zone (DMZ), Spanning Tree (especially for hierarchical network), port or network interface controller (NIC) channel, security (both poll security and physical security), wireless access points, and wireless access controllers.

Even when the design and implementation are well planned and executed, one has to be

constantly vigilant for attacks and continuously upgrade to better systems, devices and tools.

7.7. *Internet Protocol Suite* [24*, C3]

Data is transmitted in packets from one computer to another, either in the same network or in a different one. The Internet Protocol suite, or TCP/IP, defines data communication between two computers connected via the internet. The top three layers of the OSI model (Application, Presentation and Session layers) are merged into the application layer, and the network layer is revised specifically for internet functioning. Internet Protocol is the fulcrum of today's internet or network layer.

Multiple variations of Internet Protocols are designed and used for different purposes. The protocols include TCP/IP (Transmission Control Protocol/Internet Protocol), UDP/IP (User Datagram protocol / Internet Protocol), SMTP (Simple Mail Transfer Protocol), PPP (Point to Point Protocol), FTP File Transfer Protocol, SFTP (Secure FTP), HTTP (Hyper Text Transfer Protocol), HTTPS (HTTP Secure), Telnet (Terminal Network), PoP3 (Post office Protocol 3), VOIP (Voice over Internet Protocol), SLIP (Serial Line Internet Protocol). It is important to know the differences between these along with use cases (applications where each type is used or where it works best).

Mobile Internet Protocol is a communications protocol that conforms to an IETF (Internet Engineering Task Force) standard and allows users to move their mobile devices (laptops, mobile phones, etc.) seamlessly from one network to the other without changing the IP address.

Internet Protocol Version 4 (IPv4) uses a 32-bit IP address, whereas IPv6 uses 128-bit IP addresses.

Private IP addresses are translated into public IP addresses using either NAT (network address translation) or PAT (port address translation). Both use IPv4, but PAT uses port numbers. Different technologies used to communicate between IPv4 and IPv6

devices include dual-stack routers, tunneling and NAT protocol translators.

Professional computer network architects and programmers need to understand IPv6 addressing, routing, transitioning to IPv6 from IPv4, dual-dress stacks, tunneling and NAT64.

7.8. *Wireless and Mobile Networks* [c24*, C7]

Wireless networks provide the ability for devices to connect and communicate without the hassle of wires and cables. They also provide flexibility and ease of using the devices. Different wireless technologies are used for different applications:

- Wireless personal area networks (WPAN).
- Wireless local area networks (WLAN) .
- Wireless wide area networks (WWAN).

A mobile or cellular network is a radio network spread over a specific area of land (called a cell). The cells are served by base stations, which are fixed-location transceivers. To avoid interference and ensure guaranteed bandwidth, the adjacent cells use a different set of frequencies. These cells, when connected, provide wide area radio coverage. The cell patterns take different shapes, but squares, circles and hexagons are typical.

Different methods of data transmission are used between channels, such as frequency division multiple access (FDMA), time division multiple access (TDMA), code division multiple access (CDMA), space division multiple access (SDMA), etc.

Wireless technology has evolved over several generations. Software Engineers are encouraged to learn the differences among 1G, 2G, 3G, 4G and 5G technologies, along with the core network, access system, frequency, bandwidth and technologies used in each.

7.9. *Security and Vulnerabilities* [24*, C9]

Although wireless technology provides the ease of connecting seamlessly to the network, it is also prone to attacks unless the network is secured. Risks to unsecured wireless networks

include Piggybacking, Wardriving, Evil Twins attacks, Wireless sniffing, Unauthorized computer access, Shoulder sniffing and Theft of mobile devices.

Communication over the internet via mobile device is highly vulnerable to cyber-attacks. In addition to wardriving, mentioned above, typical wireless and mobile device attacks include SMiShing, War driving, WEP attacks, WPA attacks, Bluejacking, Reply attacks, Blue snarfing, RF Jamming, etc.

Many precautionary measures must be implemented and strictly followed to reduce such risks. These measures include changing default passwords, changing passwords frequently, restricting access to authorized users, encrypting data in the system and on the network, and installing multiple levels of firewalls. In addition, users must protect and hide (not publicize) service set identifier (SSID), use effective antivirus software, and update and upgrade it regularly; use a virtual private networks (VPN), use file-sharing or system-sharing access with care, and disable access after use; and update or upgrade the access point or access controller, gateway and other devices with security patches when they become available.

8. User and Developer Human Factors

The thought processes and behaviors of software developers typically differ from that of software users. This content area identifies salient parameters that matter for end users as well as the perspective of the developers. Human-computer interface (HCI) focuses on designing and developing computer technology for users to interact with computing systems.

User satisfaction is measured in terms of user experience (UX). An ideal interface would facilitate interaction that is as natural as the interaction between two human beings.

8.1. User Human Factors [3*, C8]

Users expect software to be robust; to have an intuitive graphical user interface (GUI) that

guides the user through minimal, intelligent, easy-to-follow steps to achieve the end result; to be secure; and to provide fast, consistent responses.

The interface should help users use the system easily. The interface should be self-explanatory and enable self-learning. The messages, whether communicating results or errors, should be clear and complete. The system should be able to regain its original state if there are errors.

The system should allow users to interrupt during the processing and undo the operation, wherever possible.

The software engineer needs to identify the profile of users the system; system's functionality, input and output interfaces users use (keyboard, touch pad, audio, video, etc.) to interact with the system, the system's fault tolerance, the system's performance parameters. among others.

Typically, user interface development goes through several iterations, starting with a prototype. The user interface devices must be robust.

8.2. Developer Human Factors [3*, C31 - C32]

The software lives much longer than the time taken to develop. Invariably, the software engineers who maintain the code are different from those who develop. Hence, the code has to be written with more care and for use by other programmer / software engineer.

Meaningful and comprehensive documentation is crucial at all stages of software lifecycle.

Defining and adopting apt coding standard for the project, and ensuring every team member implements the same in spirit is key for developing clean code that lives longer with minimal maintenance.

Programming style is another key ingredient of a good code. Code has to be legible, should be like reading a good poem and easily comprehensible. Using meaningful, consistent and detailed comments is essential to ensure code readability.

Other traits of a good software programmer include being a team player, enjoy

solving puzzles creatively, be agile, be structured / modular among others.

Good coding standards include defining naming conventions for various types of variables, functions/procedures, comment structure/styles, indentation styles, structuring the code into paragraphs (of related functions), etc.

“Code is read many more times than it is written. Consider whether write-time convenience is a false economy” — Steve McConnell

“Clean code always looks like it was written by someone who cares” — Robert (Uncle Bob) Martin

9. Artificial Intelligence and Machine Learning [17*]

Intelligence is the ability to acquire and correlate information and knowledge to make a correct decision for a specific task. *Artificial intelligence (AI)* enables computer systems to become intelligent, like human beings. *Machine learning (ML)* enables computer systems to learn from experiences and to use the knowledge gained to make smart decisions — to become artificially intelligent. Deep learning uses artificial neural network models for learning and making predictions.

Everyone expects all systems they use to be smart, reliable, consistent, secure and fault-tolerant — and to get better every day. AI and ML work toward enabling systems to accomplish all this.

An ideal AI system would be one that a human could not identify it as a computer; humans would not be able to distinguish the computer from a human being.

Several tools have been developed and are available for creating AI systems. Using proven tools helps engineers build a stable system faster.

9.1. Reasoning

Reasoning means analyzing sets of information available for a given situation and determining the cause of the situation. Reaching this conclusion is an important ability of AI,

as the conclusion informs AI’s decision about what to do next.

Different types of reasoning used in AI include the following:

Deductive Reasoning is a standard and strategic approach to mapping available facts, information and knowledge to arrive at a conclusion. In this approach, available facts and information are considered to be authentic. For example, if the premises are “All girls are beautiful” and “Michu is a girl,” then the conclusion is “Michu is beautiful.”

Inductive Reasoning is about introducing a hypothesis and creating generalizations from the available facts and premises. Unlike deductive reasoning, in inductive reasoning, even if the premises are certain, the conclusion would be *probable*, depending on whether the inductive argument is strong or weak. For example, check the location of all engineers working on a project and if they are from Bengaluru, India state “All employees working on the gaming project are from Bengaluru.”

Abductive Reasoning starts with an incomplete set of data or information and proceeds to derive the most likely conclusion from the latest data. For example, a doctor analyzes the latest lab reports of a patient to predict the course of the disease.

Common Sense Reasoning makes inferences about situations based on similar past experiences. For example, if a motorcycle skids while driving on a wet road, that information is remembered and considered during future rides.

Monotonic Reasoning occurs when the conclusion remains permanent or constant after it is reached. For example, “The Himalayas are one of the tallest mountain ranges.”

Non-Monotonic Reasoning (NMR) occurs when the inference changes values or direction based on new knowledge or information. NMR is based on assumptions and deals with

incomplete or not-known facts. For example, the rule is “Birds fly”. But a few birds do not fly including penguins.

Software engineers are encouraged to learn other reasoning methods, such as metalevel reasoning, procedural numeric reasoning, and formal reasoning, as well.

9.2. *Learning*

We learn from our observations, experiments and experiences. Enabling computers to learn and to remember what they’ve learned for future use is critical for building AI systems. An AI system learns when observations and outcomes of experiments (signals) are fed back into the system. Different types of learning include the following:

Supervised Learning, the computer system trains by receiving labeled (i.e., training) data. Subsequently, when any input is provided, the system compares it with the data it was trained on and generates output. Naturally, the more training data, the better the outcome. Supervised learning uses multiple learning techniques, including the classification technique and the regression technique. Supervised learning may not be able to handle complex tasks.

Unsupervised Learning, labeled or training data is not provided to the system. The system has to figure out common patterns from the input given and make inferences. The data is analyzed in real time.

Semi-supervised Learning, the system is trained with partly labeled and partly unlabeled data. This type of learning has been shown to be effective.

Reinforcement Learning is based on interactions with the environment. In this type of learning, the system receives feedback (an error message or a reward) and learns from that feedback. No data is provided to the system (neither labeled nor unlabeled). Various algorithms are produced in reinforced

learning. This is a trial-and-error method for learning.

Software engineers working on AI are expected to know various other learning techniques as well, including dimensionality reduction learning, self-learning, feature learning, sparse learning, anomaly detection and robot learning, along with the key differences between the methods and the applications where each method works well.

9.3. *Models*

AI models are inference engines or tools (algorithms) that can arrive at the best decisions based on relevant data.

Different models are created to enable efficient ML, with or without training data. Models used in ML include the following:

Linear Regression model is based on supervised learning, where the relationship between input and output variables is determined and used. This model is commonly used in health care and banking applications.

Logistic Regression model is a statistical model primarily used for classifying dependent variables from given independent variables.

Artificial Neural Networks are inspired by biological neural networks in a brain. The systems are designed to learn naturally from the inputs without specific rules.

Decision Tree model is used where past decisions are used to arrive at a decision. The name “tree” is used because the data is stored in the form of a tree.

Naïve Bayes model works on the assumption that the presence of a feature does not depend on the presence of any other feature. Spam filtering is one of the applications that suits this model.

Support Vector Machine (SVM), is a supervised ML algorithm used to analyze a limited quantum of data. SVM is typically faster than

artificial neural networks because it works with limited data.

Random Forest model uses multiple decision trees for making a final decision. The random forest model is useful for solving both regression and classification problems.

AI models are key to making the most appropriate decisions. As different models suit specific applications or domains, software engineers are encouraged to learn many other AI models as well, such as Linear Discriminant Analysis, Learning Vector Quantization, K-nearest Neighbors (KNN), etc.

9.4. Perception and Problem-Solving

Solving a problem efficiently and quickly is the goal of AI. Problem-solving predominantly comprises understanding user commands and executing them, as humans do. Depending on the application and problem to be solved, AI systems use the relevant knowledge base and predicate logic to identify the most appropriate solution.

AI systems dealing with the external world, obtain environmental data through sensors (cameras; microphones; temperature, pressure and light sensors, etc.), analyzes the data using its knowledge base or inference engine, and acts upon it.

Based on capabilities and functionality, AI systems are categorized into multiple types.

Type I AI systems are designed to do specific tasks with intelligence. Examples include Chess games, speech and image recognition, among others.

Type II AI systems analyze the current situation or environment and do not normally refer to previous decisions made in a similar situation to arrive at an appropriate action. Reactive systems or reactive machines typically make decisions and execute commands at that instance, referring to the existing knowledge base. A good example is a self-driving cars.

Type III, or self-aware, AI systems have consciousness and are mindful. These systems adopt the mind theory and predict the mood of the other person or entity based on the

person's action or type of action. For example, if the driver in the vehicle behind the system honks, then the AI system might conclude that the driver is angry or unhappy. Social and ethical behavior is part of conscious systems.

9.5. Natural Language Processing

Natural language processing (NLP) is a crucial part of AI systems, enabling users to interact with the AI systems in a way that is similar to how they interact with other humans. AI systems understand human languages and execute commands delivered in those languages. AI systems that work on voice commands need to understand not only the human language, but also the slang or pronunciation of the user.

9.6. AI and Software Engineering

Software engineering and AI are mutually related to each other in basically two ways: AI applications in software engineering (i.e., AI for SE) and software engineering for AI systems (i.e., SE for AI).

AI for SE aims to establish efficient ways of building high-quality software systems by replicating human developers' behavior. It ranges over almost all development stages, from resolving ambiguous requirements to predicting maintainability, particularly well applied in software quality assurance and analytics, such as defect prediction, test case generation, vulnerability analysis, and process assessment [15]. Although human-centric software engineering activities benefit, engineers should be aware of limitations and challenges inherent to the nature of AI and ML, especially the uncertain and stochastic behavior and the necessity of sufficiently labeled and structured datasets [15].

The development of AI systems is different from traditional software systems since the rules and system behavior of AI systems are inferred from training data rather than written down as program code [16]. Thus, there is a need for particular support of SE for AI, such as interdisciplinary collaborative teams of data scientists and software engineers, software

evolution focusing on large and changing datasets, and ethics and equity requirements engineering [16]. Recommended software

engineering practices for AI are often formalized as patterns, such as ML software design patterns [17].

MATRIX OF TOPICS VS. REFERENCE MATERIAL

Topics	Tanenbaum, Bos [19*]	CLRS [18*]	H.Washizaki [17*]	Horowitz et al. 2007 [5*]	SMcConnell [3*]	Sommerville 2016 [6*]	L. Null and J. Lobur [8*]	Articles and Journals	J.G. Brookshear [4*]	Thomas Connolly, Carolyn Begg [22]	Kurose & Ross [24]
1. Basic Concept of a System or Solution						C10					
2. Computer Architecture and Organization											
<i>2.1 Computer Architecture</i>							C1.1				
<i>2.2 Types of Computer Architecture</i>							C4.14, C5				
<i>2.2.1 Von Neumann Architecture</i>							C1.9				
<i>2.2.2 Harward Architecture</i>								[20]			
<i>2.2.3 Instruction Set Architecture</i>							C4.8.3				
<i>2.2.4 Flynn's Architecture or Taxonomy</i>							C9.3				
<i>2.2.5 System Architecture</i>						C6			C6		
<i>2.3 Micro Architecture or Computer Organization</i>							C4				
<i>2.3.1 Arithmetic Logic Unit</i>							C1.2				
<i>2.3.2 Memory Unit</i>							C6				
<i>2.3.3 Input / Output Unit</i>							C7				

[illegible]

4.4 Object-Oriented Programming									C6.5		
4.5 Distributed Programming and Parallel Programming									C6.6		
4.6 Debugging						C2.2.2					
4.7 Standards and Guidelines					C28.5, C31.5						
5. Operating Systems											
5.1 Processor Management	c2, c8										
5.2 Memory Management	c3										
5.3 Device Management	c5										
5.4 Information Management	c4										
5.5 Network Management									C4.1		
6. Database Management											
6.1 Schema										C2.1.4	
6.2 Data Models and Storage Models										C2.3	
6.3 Database Management Systems										C1.3	
6.4 Relational Database Management Systems and Normalization										C4	
6.5 Structured Query Language										C6, C7, C8	
6.6 Data Mining and Data Warehousing										C3.4	
6.7 Database Backup and Recovery										C2.2	
7. Computer Networks and Communications									C4.1		C1
7.1 Types of Computer Networks									C4.1		C1.2.1

[illegible]

REFERENCES

- [1] Joint Task Force on Computing Curricula, IEEE Computer Society and Association for Computing Machinery, Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering, 2014; <http://sites.computer.org/ccse/SE2004Volume.pdf>.
- [2*] G. Voland, *Engineering by Design*, 2nd ed., Prentice Hall, 2003.
- [3*] S. McConnell, *Code Complete*, 2nd ed., Microsoft Press, 2004.
- [4*] J.G. Brookshear, *Computer Science: An Overview*, 12th ed., Addison-Wesley, 2017.
- [5*] E. Horowitz et al., *Computer Algorithms*, 2nd ed., Silicon Press, 2007.
- [6*] I. Sommerville, *Software Engineering*, 10th ed., Addison-Wesley, 2016.
- [7] ISO/IEC/IEEE, “ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary,” 2nd ed. 2017.
- [8*] L. Null and J. Lobur, *The Essentials of Computer Organization and Architecture*, 5th ed., Jones and Bartlett Publishers, 2018.
- [9*] J. Nielsen, *Usability Engineering*, Morgan Kaufmann, 1994.
- [10] ISO 9241-420:2011 *Ergonomics of Human-System Interaction*, ISO, 2011.
- [11*] M. Bishop, *Computer Security: Art and Science*, 2nd ed, Addison-Wesley, 2018.
- [12] R.C. Seacord, *The CERT C Secure Coding Standard*, Addison-Wesley Professional, 2016.
- [13] R. Fagin, “A Normal Form for Relational Databases that is based on Domains and Keys,” *ACM Transactions on Database Systems*, Vol. 6, No. 3, ACM, September 1981
- [14] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning (Adaptive Computation and Machine Learning series)* Illustrated Edition, 2018.
- [15] S. Shafiq, A. Mashkooor, C. Mayr-Dorn, A. Egyed, “A Literature Review of Using Machine Learning in Software Development Life Cycle Stages,” *IEEE Access*, Volume 9, IEEE, October 2021.
- [16] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, “Software Engineering for AI-Based Systems: A Survey,” *ACM Transactions on Software Engineering and Methodology*, Vol. 31, No. 2, ACM, April 2022.
- [17] H. Washizaki, F. Khomh, Y. G. Gueheneuc, H. Takeuchi, N. Natori, T. Doi, S. Okuda, “Software Engineering Design Patterns for Machine Learning Applications,” *Computer*, Vol. 55, No. 3, IEEE Computer Society, March 2022.
- [18] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, Clifford Stein, “Introduction to Algorithms,” Fourth Edition, 2022.
- [19] Andrew W Tanenbaum, Herbert Bos, “Modern Operating Systems,” 4e, 2016.
- [20] <https://ieeexplore.ieee.org/document/9779481>
- [21] Neal Ford, Mark Richards, Pramod Sadalage and Zhamak Dehgh, *Software Architecture: The Hard Parts*, O Reilly, First Edition – 2021

- [22] Thomas Connolly, Carolyn Begg,
*Database Systems - A Practical Approach to
Design, Implementation and Management*,
6th Edition – Pearson
- [23] Michael J. Hernandez, Database
Design For Mere Mortals, 4th Edition,
Addison-Wesley
- [24] James F Kurose, Keith W Ross,
*Computer Networking - A Top-Down
Approach*, 7th Edition, Pearson